

SPU2D Bin Conflicts:  
Empacotamento Bidimensional em Faixas com Restrições de  
Conflitos

Ycaro Sales

3 de julho de 2026

## Resumo

Este trabalho aborda o Problema de Empacotamento Bidimensional em Faixas com Restrições de Conflitos (*2D Strip/Bin Packing with Conflicts*), uma variante NP-difícil do empacotamento clássico em que pares de itens incompatíveis não podem ser alocados em um mesmo recipiente. O objetivo é empacotar todos os itens retangulares no menor número possível de faixas de dimensões fixas, respeitando a ausência de sobreposição e o grafo de conflitos. Apresentamos uma formulação do problema, uma representação da solução baseada em chaves aleatórias e duas estratégias de resolução: uma heurística construtiva gulosa que combina coloração do grafo de conflitos com *First-Fit-Decreasing* por classe de cor, e uma metaheurística *Biased Random-Key Genetic Algorithm* (BRKGA). Os experimentos sobre 60 instâncias de três conjuntos de teste mostram que o BRKGA reduz o número total de faixas de 293 para 262 (redução média de aproximadamente 10,6%), superando a heurística gulosa em 30 instâncias e empatando nas outras 30, sem nunca produzir resultado pior.

# Sumário

|          |                                                                                                               |           |
|----------|---------------------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução</b>                                                                                             | <b>2</b>  |
| <b>2</b> | <b>Caracterização do problema</b>                                                                             | <b>3</b>  |
| 2.1      | O problema do empacotamento bidimensional em faixa com restrições de descarregamento e de conflitos . . . . . | 3         |
| <b>3</b> | <b>Fundamentação teórica</b>                                                                                  | <b>4</b>  |
| <b>4</b> | <b>Metodologia</b>                                                                                            | <b>5</b>  |
| 4.1      | Visão geral . . . . .                                                                                         | 5         |
| 4.2      | Representação da solução . . . . .                                                                            | 5         |
| 4.3      | Heurística gulosa (coloração + <i>First-Fit-Decreasing</i> ) . . . . .                                        | 5         |
| 4.4      | Metaheurística BRKGA . . . . .                                                                                | 6         |
| <b>5</b> | <b>Resultados</b>                                                                                             | <b>8</b>  |
| <b>6</b> | <b>Conclusão</b>                                                                                              | <b>11</b> |

# Capítulo 1

## Introdução

A logística ocupa papel central na economia contemporânea: o transporte e o armazenamento de mercadorias representam parcela expressiva dos custos operacionais de empresas de manufatura, varejo e comércio eletrônico. Nesse contexto, problemas de *corte e empacotamento* (*cutting and packing*) surgem sempre que é preciso acomodar um conjunto de objetos dentro de recipientes maiores — caixas em paletes, itens em contêineres, peças em chapas de matéria-prima — de forma a aproveitar ao máximo o espaço disponível. Cada recipiente economizado significa menos viagens, menos área de estoque e menor desperdício de material, com impacto direto sobre custos e sustentabilidade.

Este trabalho trata de uma generalização do empacotamento bidimensional em que, além das restrições geométricas usuais, existe a noção de *conflito* entre itens. Dois itens em conflito não podem compartilhar o mesmo recipiente. Esse tipo de restrição modela situações reais como produtos químicos incompatíveis, mercadorias destinadas a clientes ou rotas distintas, ou itens que, por razões de segurança ou regulação, não podem ser transportados juntos. O acréscimo dos conflitos torna o problema substancialmente mais difícil, pois reduz a liberdade de combinação dos itens e tende a aumentar o número de recipientes necessários.

O objetivo deste trabalho é minimizar o número de faixas (recipientes) utilizadas para empacotar todos os itens de uma instância. Para isso, implementou-se em Rust um *solver* que oferece duas abordagens: uma heurística construtiva gulosa, usada como linha de base, e uma metaheurística evolutiva (BRKGA), descritas no Capítulo 4. Os demais capítulos estão organizados da seguinte forma: o Capítulo 2 caracteriza formalmente o problema; o Capítulo 3 apresenta a fundamentação teórica; o Capítulo 4 descreve a metodologia, incluindo a representação da solução e as heurísticas empregadas; e o Capítulo 5 reporta os resultados experimentais.

## Capítulo 2

# Caracterização do problema

**Entrada:** um conjunto de faixas (recipientes) retangulares de largura fixa  $W$  e altura fixa  $H$ ; um conjunto finito  $\mathfrak{R} = \{r_1, r_2, \dots, r_n\}$  de  $n$  retângulos, representando os itens; e um grafo simples de conflitos  $G = (\mathfrak{R}, E)$ , com  $E \subseteq \{\{u, v\} \mid u, v \in \mathfrak{R} \wedge u \neq v\}$ . Cada item  $r_i$  possui largura  $w_i$ , altura  $h_i$  e classe de cliente  $c_i$ .

**Objetivo:** empacotar todos os itens  $r \in \mathfrak{R}$  dentro das faixas, minimizando o número de faixas utilizadas.

**Restrições:**

- Todos os itens devem ser empacotados;
- não são permitidas sobreposições entre itens em uma mesma faixa;
- cada item deve estar inteiramente contido nos limites da faixa que o contém ( $0 \leq x$ ,  $0 \leq y$ ,  $x + w_i \leq W$ ,  $y + h_i \leq H$ ), admitindo-se rotação de  $90^\circ$ ;
- a restrição de conflitos deve ser satisfeita: para toda aresta  $\{u, v\} \in E$ , os itens  $u$  e  $v$  não podem ser alocados em uma mesma faixa.

### 2.1 O problema do empacotamento bidimensional em faixa com restrições de descarregamento e de conflitos

O problema estudado é uma variante do empacotamento bidimensional em faixas (*2D Strip/Bin Packing*). Na formulação clássica, busca-se posicionar retângulos sem sobreposição dentro de recipientes de dimensão fixa, minimizando o número de recipientes. Aqui, a essa estrutura acrescentam-se: (i) a *restrição de conflitos*, modelada pelo grafo  $G$ , segundo a qual itens ligados por uma aresta não podem coexistir no mesmo recipiente; e (ii) a informação de *classe de cliente*  $c_i$  de cada item, que dá suporte a uma *restrição de descarregamento* — a ordem em que os itens de diferentes clientes são acomodados pode condicionar a viabilidade prática do descarregamento ao longo de uma rota. Neste trabalho, o foco recai sobre a restrição de conflitos e sobre a minimização do número de faixas; a classe de cliente é representada nos dados de entrada e fica disponível para extensões futuras.

## Capítulo 3

# Fundamentação teórica

O empacotamento bidimensional pertence à família de problemas de corte e empacotamento e é NP-difícil, generalizando o *Bin Packing* unidimensional. A introdução do grafo de conflitos aproxima o problema de formulações de coloração de grafos: se a geometria fosse ignorada, alocar itens a recipientes respeitando os conflitos equivaleria a colorir  $G$  com o menor número de cores, em que cada cor representa um recipiente. Assim, o número cromático de  $G$  é um limitante inferior para o número de faixas necessárias, pois itens mutuamente em conflito (uma clique em  $G$ ) exigem recipientes distintos. Essa conexão não é apenas um limitante teórico: ela pode ser explorada de forma *construtiva*. Ao colorir  $G$ , cada classe de cor é um conjunto independente — portanto *livre de conflitos* — e pode ser empacotada isoladamente, eliminando a restrição de conflitos por construção. Uma heurística clássica de coloração é a *coloração gulosa* em ordem de Welsh–Powell [4]: os vértices são processados por grau decrescente e cada um recebe a menor cor não usada por seus vizinhos já coloridos, o que tende a produzir poucas cores a baixo custo computacional.

Como o problema é intratável na prática para instâncias de tamanho realista, recorre-se a métodos aproximados. As *heurísticas construtivas* produzem uma solução em uma única passagem, tomando decisões locais (por exemplo, regras de ajuste como *First-Fit* e *Best-Fit*); são rápidas, mas podem ficar presas em soluções de qualidade limitada. As *metaheurísticas*, por sua vez, exploram o espaço de soluções de forma iterativa e estocástica, buscando escapar de ótimos locais. Entre elas, os algoritmos genéticos evoluem uma população de soluções por meio de seleção, recombinação (*crossover*) e mutação.

O **BRKGA** (*Biased Random-Key Genetic Algorithm*) é uma metaheurística genética em que cada solução é codificada por um vetor de chaves aleatórias reais no intervalo  $[0, 1]$ . Um *decodificador* determinístico, específico do problema, traduz esse vetor em uma solução concreta e calcula seu custo. Essa separação entre representação (genérica) e decodificação (específica) permite reutilizar todo o maquinário evolutivo — seleção, *crossover* e mutação operam apenas sobre as chaves — enquanto o conhecimento do domínio fica concentrado no decodificador.

Para o posicionamento geométrico dos itens dentro de cada recipiente, utiliza-se a técnica de *Espaços Máximos Vazios* (*Empty Maximal Spaces*, EMS): o espaço livre de um recipiente é representado por um conjunto de retângulos vazios maximais; ao inserir um item, os espaços que ele intercepta são recortados em novos retângulos e os espaços contidos em outros são descartados, mantendo apenas os maximais.

## Capítulo 4

# Metodologia

### 4.1 Visão geral

O *solver* foi implementado em Rust. Cada instância é lida de um arquivo JSON (formatos `Objects/Items` e `Container/ItemTypes`), construindo-se a estrutura `Instance` com os itens, as dimensões da faixa e o grafo de conflitos indexado pela posição de cada item. Para cada instância são executadas duas estratégias — a heurística gulosa e o BRKGA — e as soluções resultantes são serializadas em JSON, registrando o número de faixas utilizadas, os itens não alocados, o tempo de execução e a posição de cada item.

### 4.2 Representação da solução

A solução de uma instância é o conjunto de faixas, cada uma contendo as posições (origem, dimensões e orientação) dos itens nela alocados. O componente geométrico central é o `Container`, que mantém o conjunto de Espaços Máximos Vazios (EMS) e as colocações já realizadas. A inserção segue a regra *bottom-left*: dentre os espaços livres que comportam o item, escolhe-se o de menor origem (primeiro em  $y$ , depois em  $x$ ); a checagem de conflito rejeita o item caso algum vizinho seu em  $G$  já esteja na faixa.

No BRKGA, a solução é codificada por um vetor de  $2n$  chaves aleatórias em  $[0, 1]$ , interpretado da seguinte forma:

- as primeiras  $n$  chaves definem a **ordem de inserção** dos itens: os itens são ordenados de forma crescente pela sua chave;
- as últimas  $n$  chaves definem a **orientação** de cada item: uma chave maior ou igual a um limiar (0,5) indica que o item é rotacionado em  $90^\circ$ ; caso contrário, mantém a orientação original.

O decodificador percorre os itens na ordem dada e os insere por *First-Fit* com orientação fixada, abrindo uma nova faixa quando nenhuma das abertas aceita o item; um item que não cabe nem em uma faixa vazia na sua orientação forçada é marcado como não alocado.

### 4.3 Heurística gulosa (coloração + *First-Fit-Decreasing*)

A heurística gulosa serve de linha de base e combina coloração do grafo de conflitos com empacotamento *First-Fit-Decreasing*. Primeiro,  $G$  é colorido por um algoritmo guloso em ordem de

Welsh–Powell (grau decrescente, com desempate determinístico): itens em conflito recebem cores diferentes, de modo que cada *classe de cor* é um conjunto livre de conflitos. Em seguida, os itens são agrupados por cor e cada grupo é empacotado separadamente: os itens da cor são ordenados por área decrescente e inseridos por *First-Fit* em *faixas dedicadas àquela cor*, abrindo-se uma nova faixa da mesma cor quando nenhuma das abertas aceita o item.

Como as faixas nunca misturam cores, os conflitos tornam-se impossíveis *por construção*, sem depender da checagem de vizinhança a cada inserção. A geometria permanece a mesma do restante do *solver*: gerenciamento de espaços livres por EMS, regra *bottom-left* e tentativa automática das duas orientações do item. O Algoritmo 1 resume o procedimento.

---

**Algoritmo 1:** Heurística gulosa: coloração + First-Fit-Decreasing por cor

---

**Input:** instância com itens  $\mathcal{R}$ , faixa  $W \times H$ , grafo  $G$

**Output:** conjunto de faixas  $B$

---

```

1  $B \leftarrow \emptyset$ ;
2 cora  $G$  gulosamente em ordem de Welsh–Powell (grau decrescente);
3 agrupe os itens por cor;
4 foreach classe de cor  $C$  do
5      $B_C \leftarrow \emptyset$ ; // faixas dedicadas à cor  $C$ 
6     ordene os itens de  $C$  por área decrescente;
7     foreach item  $r \in C$  do
8          $alocado \leftarrow falso$ ;
9         foreach faixa  $b \in B_C$  do
10            if  $b$  tem espaço para  $r$  then
11                insira  $r$  em  $b$  (regra bottom-left, auto-rotação);
12                 $alocado \leftarrow verdadeiro$ ; break;
13            end
14        end
15        if  $\neg alocado$  then
16            abra nova faixa  $b'$  e insira  $r$  em  $b'$ ;
17             $B_C \leftarrow B_C \cup \{b'\}$ ;
18        end
19    end
20     $B \leftarrow B \cup B_C$ ;
21 end
22 return  $B$ ;

```

---

## 4.4 Metaheurística BRKGA

A metaheurística evolui uma população de vetores de chaves aleatórias. A *função de aptidão* (*fitness*) de um cromossomo é obtida decodificando-o e calculando

$$f = \text{faixas\_usadas} + \text{itens\_nao\_alocados} \cdot (n + 1),$$

a ser *minimizada*. A penalidade  $(n + 1)$  por item não alocado garante que qualquer solução inviável seja sempre pior do que qualquer solução viável (que usa, no máximo,  $n$  faixas), de modo que a busca prioriza a viabilidade. O processo evolutivo, ilustrado no Algoritmo 2, foi configurado com



população de 100 indivíduos, critério de parada de 200 gerações sem melhora, seleção por torneio, *crossover* uniforme e mutação de gene único.

---

**Algoritmo 2:** BRKGA para o empacotamento com conflitos

---

**Input:** instância, tamanho de população  $P$ , máximo de gerações estagnadas  $S$

**Output:** melhor empacotamento encontrado

```
1 inicialize população de  $P$  vetores de  $2n$  chaves em  $[0, 1]$ ;  
2 repeat  
3   foreach cromossomo do  
4     decodifique (ordem + orientação) e empacote por First-Fit;  
5     avalie  $f = \text{faixas} + \text{nao\_alocados} \cdot (n + 1)$ ;  
6   end  
7   selecione por torneio; aplique crossover uniforme e mutação;  
8   forme a nova geração;  
9 until  $S$  gerações sem melhora;  
10 return decodificação do melhor cromossomo;
```

---

## Capítulo 5

# Resultados

As duas estratégias foram avaliadas sobre 60 instâncias amostradas de três conjuntos de teste de empacotamento bidimensional (*CJCM08*, *BPP-Subproblems* e *InterestingInstances*), com  $n$  variando de 10 a 27 itens. A cada instância original foram adicionados conflitos com três densidades distintas (sufixos *c0.1*, *c0.3* e *c0.5*, indicando a probabilidade de conflito entre pares de itens; 20 instâncias por densidade). Em todas as 60 instâncias, ambas as estratégias alocaram todos os itens. A Tabela 5.1 compara o número de faixas utilizadas pela heurística gulosa e pelo BRKGA em uma amostra representativa, junto com o total sobre o conjunto completo.

Tabela 5.1: Número de faixas utilizadas: gulosa vs. BRKGA (amostra representativa e total).

| Instância                       | $n$ | Gulosa     | BRKGA      |
|---------------------------------|-----|------------|------------|
| E00N15_c0.3                     | 15  | 4          | 3          |
| BPPSP-n100m18b2d2_Hard_c0.1     | 18  | 4          | 3          |
| BPPSP-n100m18b2d2_Hard_c0.5     | 18  | 6          | 5          |
| CJCM-E03X18_HardFeasible_c0.1   | 18  | 3          | 2          |
| CJCM-E03X18_HardFeasible_c0.3   | 18  | 4          | 4          |
| CJCM-E02F20_ManyCalls_c0.5      | 20  | 7          | 6          |
| CJCM-E02N20_Hard_c0.5           | 20  | 6          | 6          |
| E00N23_c0.3                     | 23  | 5          | 4          |
| CJCM-E00N23_HardInfeasible_c0.5 | 23  | 7          | 6          |
| BPP n24_c0.5                    | 24  | 7          | 6          |
| BPP n26_c0.5                    | 26  | 8          | 6          |
| BPP n27_c0.1                    | 27  | 4          | 3          |
| <b>Total (60 instâncias)</b>    |     | <b>293</b> | <b>262</b> |

Considerando o conjunto completo de 60 instâncias, a heurística gulosa utilizou 293 faixas no total, enquanto o BRKGA utilizou 262 — 31 faixas economizadas, uma redução média de aproximadamente 10,6%. O BRKGA produziu soluções estritamente melhores em 30 instâncias, empatou nas outras 30 e em nenhum caso foi pior do que a gulosa. Quanto ao tempo de execução, a heurística gulosa é praticamente instantânea ( $< 0,001$  s por instância), ao passo que o BRKGA, por seu caráter iterativo, gasta em média cerca de 0,10 s por instância (máximo de 0,18 s) — um custo ainda baixo, compatível com seu ganho de qualidade. A Figura 5.1 resume a comparação de qualidade sobre

todas as instâncias.

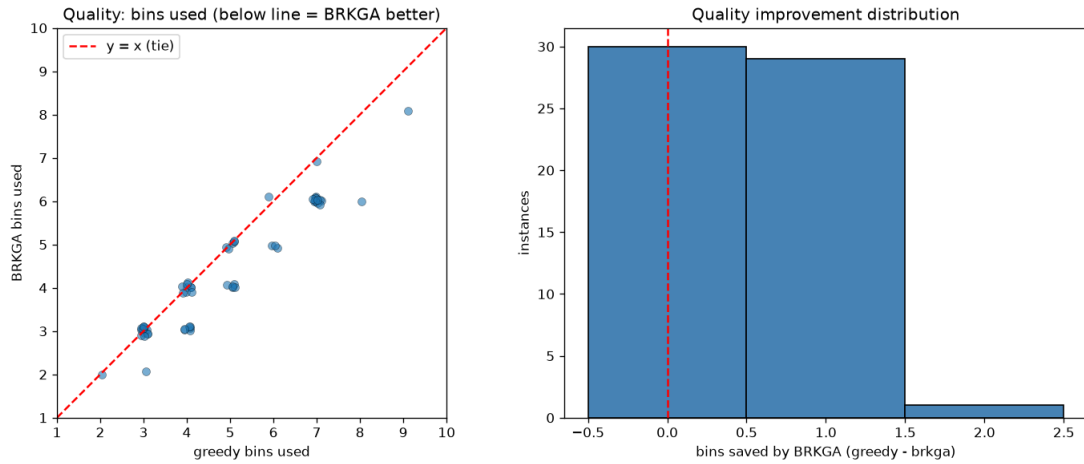


Figura 5.1: Qualidade das soluções: faixas usadas pela gulosa vs. BRKGA. Cada ponto é uma instância; abaixo da reta  $y = x$ , o BRKGA usa menos faixas. À direita, a distribuição de faixas economizadas.

Observa-se também que o aumento da densidade de conflitos (de  $c0.1$  para  $c0.5$ ) eleva consistentemente o número de faixas necessárias, o que é esperado: quanto mais arestas no grafo de conflitos, menos itens podem compartilhar um mesmo recipiente. A Tabela 5.2 agrega os resultados por densidade: nos cenários mais restritos ( $c0.5$ ) é justamente onde o BRKGA mais se distancia da heurística gulosa, evidenciando o valor da busca metaheurística quando as restrições de conflito se intensificam.

Tabela 5.2: Faixas utilizadas por densidade de conflitos (soma sobre 20 instâncias por densidade).

| Densidade | Gulosa | BRKGA | Economia  |
|-----------|--------|-------|-----------|
| $c0.1$    | 64     | 58    | 6         |
| $c0.3$    | 93     | 85    | 8         |
| $c0.5$    | 136    | 119   | <b>17</b> |

A Figura 5.2 ilustra a diferença entre as duas estratégias na instância E00N15\_ $c0.3$  (15 itens, 35 conflitos): sobre o mesmo grafo de conflitos, a gulosa usa 4 faixas e o BRKGA encontra um empacotamento com 3.

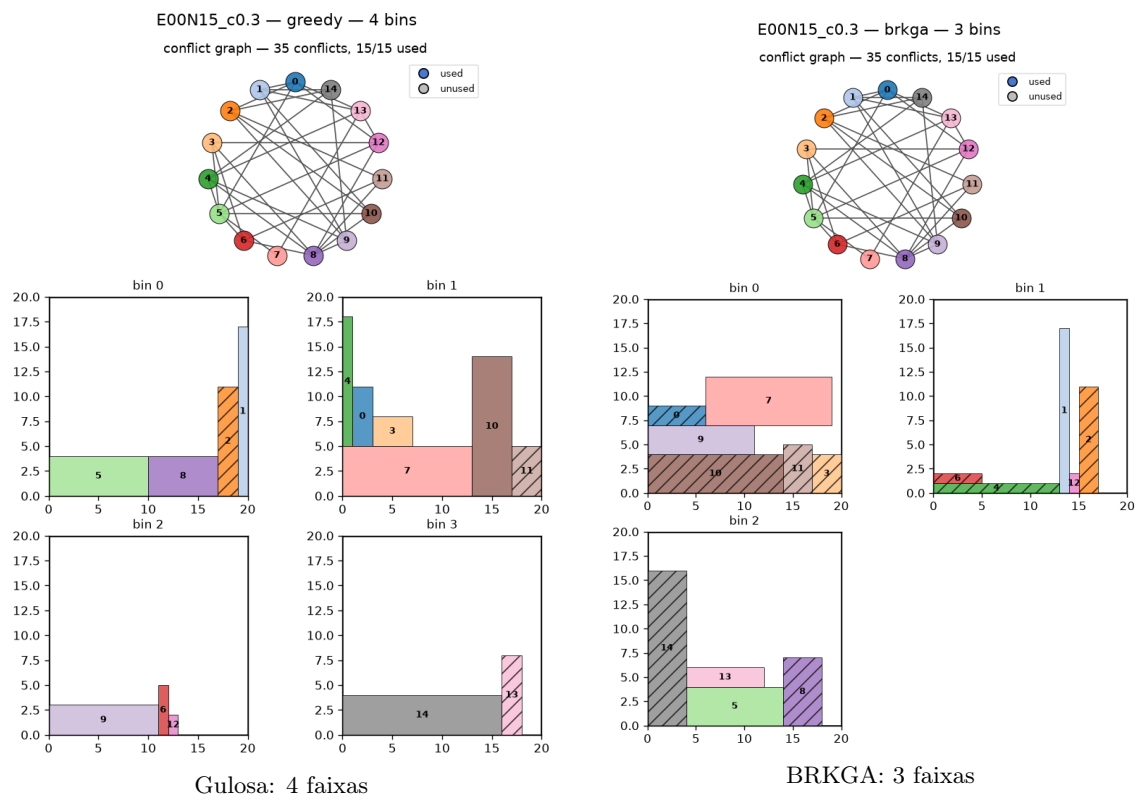


Figura 5.2: Soluções da instância E00N15\_c0.3: gulosa (esquerda) vs. BRKGA (direita).

## Capítulo 6

# Conclusão

Este trabalho apresentou uma formulação e duas estratégias de resolução para o empacotamento bidimensional em faixas com restrições de conflitos. A representação por chaves aleatórias do BRKGA, combinada a um decodificador *First-Fit* ciente de conflitos e ao gerenciamento de espaços livres via EMS, mostrou-se eficaz mesmo frente a uma linha de base fortalecida pela coloração do grafo de conflitos: sobre 60 instâncias de três conjuntos de teste, o BRKGA venceu em 30, empatou em 30 e não perdeu em nenhuma, economizando 31 faixas no total. Como trabalhos futuros, destacam-se a incorporação efetiva da restrição de descarregamento por classe de cliente, a ampliação da avaliação para todo o acervo de instâncias e o ajuste sistemático dos parâmetros da metaheurística.

## Referências Bibliográficas

# Referências Bibliográficas

- [1] Gonçalves, J. F.; Resende, M. G. C. (2011). *Biased random-key genetic algorithms for combinatorial optimization*. Journal of Heuristics, 17(5), 487–525.
- [2] Lodi, A.; Martello, S.; Monaci, M. (2002). *Two-dimensional packing problems: A survey*. European Journal of Operational Research, 141(2), 241–252.
- [3] Wäscher, G.; Haußner, H.; Schumann, H. (2007). *An improved typology of cutting and packing problems*. European Journal of Operational Research, 183(3), 1109–1130.
- [4] Welsh, D. J. A.; Powell, M. B. (1967). *An upper bound for the chromatic number of a graph and its application to timetabling problems*. The Computer Journal, 10(1), 85–86.